

PAI-1 · IntegriDos

Verificadores de Integridad en la Transmisión para SecBank

Guía de desarrollo del proyecto

Stack tecnológico, plan de trabajo, reparto de tareas por rol y anexo de configuración de GitHub para el seguimiento del equipo.

Asignatura	Seguridad en Sistemas Informáticos e Internet (SSII)
Equipo	Security Team — Grupo 20
Seguimiento	22–23 de septiembre de 2026
Entrega final	5 de octubre de 2026, 23:59h

1. Stack tecnológico

El enunciado prohíbe TLS/HTTPS/SSL en la capa de transporte: la elección real no es "qué tan segura es la conexión", sino sobre qué transporte resulta más cómodo construir, depurar y demostrar la seguridad de aplicación que exige el PAI. Se elige **API REST sobre HTTP en texto plano** (Opción B del enunciado).

Por qué REST y no sockets crudos o gRPC

- Las cabeceras HTTP (**X-Signature**, **X-Nonce**, **X-Timestamp**) separan limpiamente los metadatos de seguridad del payload de negocio: el código de verificación queda aislado en un único punto (middleware), fácil de explicar y defender.
- Las capturas de Wireshark son legibles directamente filtrando por **http**, sin necesidad de reconstruir streams (sockets) ni decodificar Protobuf (gRPC).
- Herramientas de prueba estándar (Postman, curl, httpx) generan tráfico de test sin escribir código adicional.
- No añade curva de aprendizaje ni framing manual (sockets) ni definición de esquemas .proto (gRPC): el tiempo del equipo se dedica a la parte criptográfica, que es donde está el peso real de la evaluación.

Componente	Elección	Motivo
Transporte	API REST / HTTP (FastAPI)	Opción B del enunciado; sin TLS, puerto 8080
Lenguaje	Python 3.11+	Estándar criptográfico completo en librería estándar
Derivación de claves	Argon2id (argon2-cffi)	Más moderno que PBKDF2; resistente a ataques con GPU/ASIC
Integridad/autenticidad	HMAC-SHA256 (hmac, hashlib)	Exigido por RS2; nativo del lenguaje
Generación de claves/nonces	secrets (PRNG criptográfico)	Claves de 256 bits mínimo, nonces impredecibles
Comparación segura	secrets.compare_digest()	Tiempo constante, exigido por RS4
Persistencia	SQLite (SQLAlchemy)	Suficiente para el alcance del PAI, sin infraestructura extra
Testing	pytest + matplotlib	Tests de HMAC/replay + gráfica de análisis de tiempo constante
Evidencias de red	Wireshark / tcpdump	Captura .pcap obligatoria del tráfico firmado

Todos los equipos son evaluados con el mismo rigor independientemente del transporte elegido: esta decisión no da ni resta nota por sí misma, solo cambia dónde se invierte el tiempo de desarrollo.

2. Guía de desarrollo del proyecto

2.1 Orden de implementación recomendado

No desenvolpeéis los 4 requisitos de seguridad en paralelo desde el primer día: hay una dependencia natural entre ellos. Seguid este orden:

- 1 **Base funcional** — endpoints de registro/login/logout y estructura de transacción en JSON, sin ninguna protección todavía. Sirve para que el equipo tenga algo end-to-end cuanto antes.
- 2 **RS1 (credenciales)** — sustituir el almacenamiento de contraseñas por Argon2id + salt aleatorio, y añadir el bloqueo tras N intentos fallidos.
- 3 **RS2 (integridad/autenticidad)** — firmar cada transacción con HMAC-SHA256 usando una clave de sesión de 256 bits, y verificarla en el servidor antes de procesar nada.
- 4 **RS3 (anti-replay)** — añadir nonce y timestamp a cada mensaje firmado, y la tabla de nonces vistos en el servidor. Esto depende de que RS2 ya esté firmando el mensaje completo (incluido el propio nonce).
- 5 **RS4 (tiempo constante)** — auditar TODAS las comparaciones de HMAC/tokens del código y sustituir cualquier `==` por comparación en tiempo constante. Es rápido de hacer, pero solo tiene sentido una vez existen HMACs que comparar.
- 6 **Evidencias** — capturas Wireshark de tráfico legítimo, de un ataque MitM rechazado y de un ataque Replay rechazado; y el análisis de tiempo constante.
- 7 **Documentación** — con el sistema ya funcionando y las evidencias en la mano, redactar la memoria es mucho más rápido que intentar documentar algo a medio hacer.

2.2 Reglas que no se pueden romper

- **Cero TLS/HTTPS/SSL** en ningún punto del transporte: revisad que ningún framework lo active por defecto (certificados, `ssl_context`, etc.). Está penalizado explícitamente.
- **Nunca MD5/SHA1** para contraseñas, ni contraseñas en texto plano.
- **Nunca `==`** para comparar HMACs, firmas o tokens de sesión: siempre comparación en tiempo constante.
- El servidor **solo** registra una transacción tras validar de forma estricta integridad y autenticidad — nunca "validar y seguir aunque falle".
- Toda la lógica de protección (HMAC, nonces, timing) se implementa **manualmente en capa de aplicación**, nunca delegada a una librería de transporte seguro.

2.3 Uso de IA generativa

Está permitido como asistente de programación, depuración o estructuración de la memoria, pero con tres condiciones explícitas del enunciado que hay que cumplir literalmente:

- Cada miembro debe entender y ser capaz de **defender oralmente** cualquier control criptográfico que haya en su parte del código, se haya escrito con ayuda de IA o no.
- Revisar explícitamente que el código asistido por IA no introduce TLS, hashes obsoletos ni comparadores vulnerables a timing — la IA puede sugerirlos "por buenas prácticas" sin saber que aquí están prohibidos.
- Incluir en la memoria un apartado breve indicando qué tareas se apoyaron en IA y cómo se validó la corrección del resultado.

2.4 Calendario

Fecha	Hito
22–23 septiembre	Seguimiento con el profesor. Llevad al menos la base funcional + RS1/RS2 arrancando, aunque no esté terminado.
Días intermedios	RS3, RS4, evidencias (.pcap, gráfica de timing) y redacción de la memoria.
5 octubre, 23:59	Entrega de PAI1-ST20.zip por la Plataforma de Enseñanza Virtual (nunca por email: penalización del 10%). 10% de penalización por cada día de retraso.

3. Reparto de tareas (3 roles)

Repartido por bloque de requisito, no por capa técnica, para que cada rol pueda defender su parte de forma autocontenida en la exposición. Asignad los nombres vosotros según preferencias y carga de la asignatura.

Rol A — Credenciales y acceso (RS1 + RF1)

- Registro y login de usuarios; impedir usuarios duplicados.
- Derivación de contraseñas con Argon2id y salt aleatorio único por usuario.
- Rate limiting / bloqueo temporal tras N intentos fallidos de login.
- Gestión de sesiones activas y mecanismo de logout.
- Base de datos inicial con usuarios pre-registrados para pruebas.
- Tests unitarios de la parte de credenciales.

Rol B — Integridad y anti-replay (RS2 + RS3 + RF2)

- Endpoint de transferencia y validación estricta antes de registrar cualquier transacción.
- Firma HMAC-SHA256 de cada mensaje (cliente) y verificación (servidor).
- Generación de claves de sesión de 256 bits con PRNG criptográfico.
- Generación de nonce y timestamp por mensaje; tabla de nonces procesados en el servidor.
- Rechazo de mensajes duplicados o fuera de la ventana temporal válida.
- Tests unitarios de firma HMAC y de anti-replay.

Rol C — Canal lateral, evidencias y documentación (RS4)

- Auditoría de tiempo constante: localizar y corregir cualquier comparación insegura (==) de HMACs o tokens en todo el proyecto.
- Script/experimento que mida y grafique el tiempo de verificación ante MAC correcto vs incorrecto en distintas posiciones de byte.
- Capturas Wireshark: tráfico legítimo, intento de MitM rechazado, intento de Replay rechazado.
- Diagramas de secuencia cliente-servidor para la memoria.
- Coordinar la matriz de trazabilidad de requisitos (qué fichero/línea implementa cada RS/RF).

Trabajo conjunto de los tres roles: prueba de integración end-to-end, revisión cruzada de que ninguna parte ha colado TLS o comparaciones inseguras, y ensamblaje final de la memoria (resumen ejecutivo, manual de despliegue, matriz de trazabilidad).

Anexo · Configuración de GitHub para el seguimiento

El objetivo de esta configuración es que cualquiera —incluido el profesor— pueda ver de un vistazo quién ha hecho qué y en qué requisito de seguridad, sin tener que preguntar al equipo.

A.1 Crear el repositorio

```
gh repo create secbank-integridos --private --source=. --remote=origin
git add .
git commit -m "chore: estructura inicial del proyecto"
git branch -M main
git push -u origin main
```

Añadid a los otros dos miembros como colaboradores (Settings → Collaborators), o creadlo bajo una organización con los tres como miembros: así el historial de commits queda atribuido a cada persona real, no a una única cuenta.

A.2 Estrategia de ramas

```
main      -> siempre desplegable, protegida
develop   -> rama de integración del equipo
feature/rs1-credenciales      (Rol A)
feature/rs2-rs3-hmac-nonce    (Rol B)
feature/rs4-timing-evidencias (Rol C)
```

Cada rol trabaja en su rama `feature/*` partiendo de `develop`, y abre un Pull Request hacia `develop` cuando su parte está lista. `develop` se fusiona a `main` en los puntos de integración (por ejemplo, antes del seguimiento y antes de la entrega final).

A.3 Proteger la rama main

Settings → Branches → Add rule, sobre `main`:

- Require a pull request before merging (mínimo 1 aprobación de otro miembro).
- Require status checks to pass before merging (el workflow de CI que corre los tests).
- Require branches to be up to date before merging.

A.4 Etiquetas (labels) por requisito

```
gh label create RS1 --color 1D76DB
gh label create RS2 --color 0E8A16
gh label create RS3 --color FBCA04
gh label create RS4 --color D93F0B
gh label create docs --color C5DEF5
gh label create evidencia --color 5319E7
```

Cada Issue y cada Pull Request se etiqueta con el requisito al que pertenece (RS1–RS4, docs o evidencia). Es la forma más rápida de que, en el seguimiento, podáis filtrar y enseñar "todo lo que llevamos de RS3" en un clic.

A.5 Convención de commits

```
feat(rs2): implementar firma HMAC-SHA256 en el middleware
fix(rs3): corregir ventana de expiración de nonces
test(rs4): añadir script de análisis de tiempo constante
docs: añadir diagrama de secuencia al informe
```

Prefijos: `feat`, `fix`, `test`, `docs`, `refactor`, `chore`. Referenciar el requisito en el *scope* (`rs1...rs4`) convierte el propio `git log` en un borrador casi listo de la matriz de trazabilidad de requisitos que pide

la memoria.

A.6 Tablero de seguimiento (opcional)

```
gh project create --owner --title IntegriDos
```

Tres columnas — *To do*, *In progress*, *Done* — con una tarjeta por tarea de las listadas en la sección 3. Da una vista instantánea del avance real de cada rol antes del seguimiento del 22-23 de septiembre.

A.7 Integración continua (CI) mínima

Un workflow de GitHub Actions que ejecute los tests en cada push/PR evita que una rama rompa la de otro miembro sin que nadie se entere hasta el final:

- Ejecutar pytest en cada push y en cada Pull Request.
- Comprobación automática (grep) de que no se ha colado ningún `ssl_certfile`, `ssl_keyfile` o `https://` en el código — falla la build si aparece.
- Marcar ese check como obligatorio en la protección de main (ver A.3).